

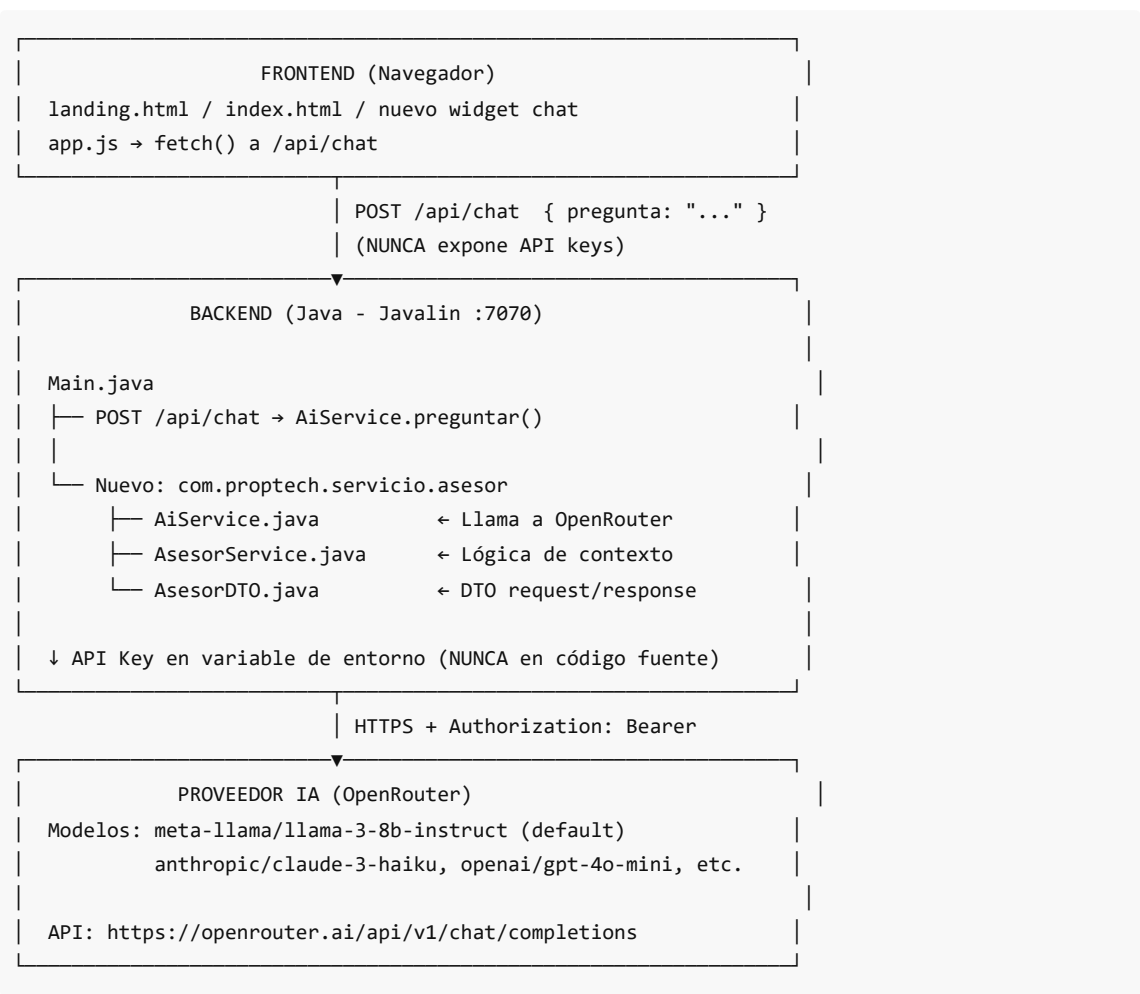
Plan de Implementación: Asesor Virtual con IA

Asistente inteligente para el sistema inmobiliario Insignia Inmo

1. Resumen

Implementar un **Asesor Virtual con IA** que permita a los usuarios del sistema inmobiliario hacer preguntas en lenguaje natural y recibir respuestas contextualizadas. El asesor podrá responder sobre propiedades, clientes, operaciones, recomendaciones, y ayudar en la gestión diaria del negocio.

2. Arquitectura Propuesta



Principio fundamental

La API Key NUNCA toca el frontend. El navegador solo se comunica con nuestro backend Java. El backend es el único que tiene acceso a la clave del proveedor de IA.

3. Nuevos Archivos a Crear

3.1 Backend (Java) — Capa Servicio

```
utilidades/src/main/java/com/proptech/servicio/asesor/
├─ AiService.java           ← Comunicación HTTP con OpenRouter
├─ AsesorService.java       ← Lógica de negocio + contexto
├─ ChatRequest.java         ← DTO para petición
└─ ChatResponse.java        ← DTO para respuesta
```

3.2 Frontend — Widget de Chat

```
utilidades/src/main/resources/public/
├─ chat-widget.html         ← HTML del widget flotante
├─ chat-widget.js           ← Lógica del chat (sin API keys)
└─ chat-widget.css          ← Estilos del widget
```

3.3 Modificaciones en Archivos Existentes

Archivo	Cambio
Main.java	Agregar endpoint POST /api/chat que recibe { pregunta } y responde { respuesta }. Inicializar AiService y AsesorService. Configurar CORS si es necesario.
index.html O landing.html	Incluir el widget de chat (<script src="/chat-widget.js">, <link href="/chat-widget.css">)
pom.xml	No requiere nuevas dependencias (usamos java.net.http.HttpClient nativo de Java 11+)

4. Diseño Detallado de Clases

4.1 AiService.java

```
package com.proptech.servicio.asesor;

import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

/**
 * Servicio que se comunica con OpenRouter API.
 * Es la ÚNICA clase que conoce la API Key.
 * La clave se lee de variable de entorno, NUNCA está hardcodeda.
 */
public class AiService {

    private static final String API_URL = "https://openrouter.ai/api/v1/chat/completions";
    private static final String MODELO_DEFAULT = "meta-llama/llama-3-8b-instruct";
    private static final int TIMEOUT_SEGUNDOS = 30;

    private final HttpClient client;
```

```

private final String apiKey;
private final String model;

public AiService() {
    this.client = HttpClient.newHttpClient();
    // ⚠ Leer de variable de entorno, NUNCA hardcodear
    this.apiKey = System.getenv("OPENROUTER_API_KEY");
    if (this.apiKey == null || this.apiKey.isBlank()) {
        System.err.println("⚠ OPENROUTER_API_KEY no configurada. " +
            "El Asesor IA no funcionará.");
    }
    this.model = MODELO_DEFAULT;
}

/**
 * Envía una pregunta al modelo y retorna la respuesta textual.
 * @param systemPrompt Personalidad del asistente
 * @param userMessage Mensaje del usuario
 * @return Respuesta del modelo
 */
public String preguntar(String systemPrompt, String userMessage)
    throws Exception {

    // 1. Construir payload JSON
    String json = construirPayload(systemPrompt, userMessage);

    // 2. Construir request HTTP
    HttpRequest request = HttpRequest.newBuilder()
        .uri(URI.create(API_URL))
        .header("Authorization", "Bearer " + apiKey)
        .header("Content-Type", "application/json")
        .POST(HttpRequest.BodyPublishers.ofString(json))
        .build();

    // 3. Enviar y recibir respuesta
    HttpResponse<String> response = client.send(request,
        HttpResponse.BodyHandlers.ofString());

    // 4. Extraer solo el texto de la respuesta del JSON
    return extraerRespuesta(response.body());
}

private String construirPayload(String systemPrompt, String userMessage) {
    // Escape de caracteres especiales para JSON
    String mensajeEscapado = userMessage
        .replace("\\", "\\\\")
        .replace("\"", "\\\"")
        .replace("\n", "\\n")
        .replace("\r", "\\r")
        .replace("\t", "\\t");

    return String.format(""""

```

```

        {
            "model": "%s",
            "messages": [
                {"role": "system", "content": "%s"},
                {"role": "user", "content": "%s"}
            ],
            "temperature": 0.7,
            "max_tokens": 512
        }
        """ , model, systemPrompt, mensajeEscapado);
    }

    private String extraerRespuesta(String jsonRespuesta) {
        // Parsear JSON para extraer:
        // choices[0].message.content
        // (usando Jackson o parse manual)
        // ...
    }
}

```

Consideraciones:

- La API Key se lee de `System.getenv("OPENROUTER_API_KEY")`
- Timeout configurable de 30 segundos
- El `systemPrompt` define la personalidad del asesor
- Se usa `HttpClient` nativo (Java 11+, no requiere dependencias externas)

4.2 AsesorService.java

```

package com.proptech.servicio.asesor;

import com.proptech.modelo.*;
import com.proptech.servicio.*;
import com.proptech.utilidades.estructuras.ListaEnlazada;

/**
 * Servicio que orquesta la conversación.
 * Construye el contexto del negocio y se lo pasa a AiService.
 * NO conoce API keys, solo se comunica con AiService.
 */
public class AsesorService {

    private final AiService aiService;
    private final InventarioInmueblesService inventarioService;
    private final ClientesService clientesService;
    private final OperacionesService operacionesService;
    // ... otros servicios según necesidad

    public AsesorService(
        AiService aiService,
        InventarioInmueblesService inventarioService,

```

```

        ClientesService clientesService,
        OperacionesService operacionesService) {
    this.aiService = aiService;
    this.inventarioService = inventarioService;
    this.clientesService = clientesService;
    this.operacionesService = operacionesService;
}

/**
 * Procesa una pregunta del usuario y retorna una respuesta contextualizada.
 */
public ChatResponse procesarPregunta(ChatRequest request) {
    try {
        // 1. Construir contexto del negocio
        String contexto = construirContexto();

        // 2. Definir la personalidad del asesor
        String systemPrompt = String.format("""
            Eres un asesor virtual experto en el sistema inmobiliario
            "Insignia Inmo". Ayudas a agentes y clientes con información
            sobre propiedades, clientes, operaciones y recomendaciones.

            Contexto actual del sistema:
            %s

            Responde de forma breve, clara y amable.
            Si no sabes la respuesta, sugiere contactar a un asesor humano.
            """, contexto);

        // 3. Enviar a la IA
        String respuesta = aiService.preguntar(systemPrompt, request.getPregunta());

        return new ChatResponse(respuesta);

    } catch (Exception e) {
        return new ChatResponse("Lo siento, ocurrió un error al procesar tu " +
            "pregunta. Por favor intenta de nuevo.");
    }
}

/**
 * Construye un resumen del estado actual del sistema
 * para que la IA pueda responder con contexto real.
 */
private String construirContexto() {
    StringBuilder sb = new StringBuilder();
    sb.append("- Total inmuebles: ").append(inventarioService.obtenerTodos().size());
    sb.append("\n- Total clientes: ").append(clientesService.obtenerTodos().size());
    sb.append("\n- Total operaciones:
").append(operacionesService.obtenerHistorial().size());
    // ... más datos relevantes
    return sb.toString();
}

```

```
}  
}
```

4.3 ChatRequest.java / ChatResponse.java

```
// ChatRequest.java  
package com.proptech.servicio.asesor;  
  
public class ChatRequest {  
    private String pregunta;  
    private String emailUsuario; // Opcional: para personalizar  
  
    public ChatRequest() {}  
  
    public String getPregunta() { return pregunta; }  
    public void setPregunta(String pregunta) { this.pregunta = pregunta; }  
    public String getEmailUsuario() { return emailUsuario; }  
    public void setEmailUsuario(String emailUsuario) { this.emailUsuario = emailUsuario; }  
}  
  
// ChatResponse.java  
package com.proptech.servicio.asesor;  
  
public class ChatResponse {  
    private String respuesta;  
    private boolean exito;  
  
    public ChatResponse() {}  
    public ChatResponse(String respuesta) {  
        this.respuesta = respuesta;  
        this.exito = true;  
    }  
  
    public String getRespuesta() { return respuesta; }  
    public void setRespuesta(String respuesta) { this.respuesta = respuesta; }  
    public boolean isExito() { return exito; }  
    public void setExito(boolean exito) { this.exito = exito; }  
}
```

5. Endpoint en Main.java

Se agregará a `Main.java` la siguiente ruta:

```
// En el método main(), dentro de la configuración de rutas:  
  
app.post("/api/chat", ctx -> {  
    // 1. Parsear request  
    ChatRequest request = ctx.bodyAsClass(ChatRequest.class);
```

```

// 2. Validar
if (request.getPregunta() == null || request.getPregunta().isBlank()) {
    ctx.status(400).json(Map.of("error", "La pregunta no puede estar vacía"));
    return;
}

// 3. Procesar con el asesor
ChatResponse response = asesorService.procesarPregunta(request);

// 4. Responder
ctx.json(response);
});

```

Configuración CORS (si el frontend está en otro puerto):

```

// En la configuración inicial de Javalin:
app.before(ctx -> {
    ctx.header("Access-Control-Allow-Origin", "*");
    ctx.header("Access-Control-Allow-Methods", "GET, POST, PUT, DELETE, OPTIONS");
    ctx.header("Access-Control-Allow-Headers", "Content-Type, Authorization");
});

// Manejar OPTIONS para preflight requests
app.options("/api/chat", ctx -> {
    ctx.header("Access-Control-Allow-Origin", "*");
    ctx.header("Access-Control-Allow-Methods", "POST, OPTIONS");
    ctx.header("Access-Control-Allow-Headers", "Content-Type");
    ctx.status(204);
});

```

6. Frontend — Widget de Chat

6.1 chat-widget.html (fragmento para integrar)

```

<!-- Widget de Asesor IA - Flotante en esquina inferior derecha -->
<div id="chat-widget" class="chat-widget">
    <!-- Botón flotante para abrir/cerrar -->
    <button id="chat-toggle" class="chat-toggle-btn">
        🗨 Asesor IA
    </button>

    <!-- Ventana de chat (oculta por defecto) -->
    <div id="chat-window" class="chat-window hidden">
        <div class="chat-header">
            <span>🤖 Asesor Virtual</span>
            <button id="chat-close">✕</button>
        </div>
        <div id="chat-messages" class="chat-messages">
            <!-- Mensaje de bienvenida -->

```

```

        <div class="message bot">
            ¡Hola! Soy el asesor virtual de Insignia Inmo.
            ¿En qué puedo ayudarte?
        </div>
    </div>
    <div class="chat-input-area">
        <input type="text" id="chat-input"
            placeholder="Escribe tu pregunta..." />
        <button id="chat-send">Enviar</button>
    </div>
    <div id="chat-typing" class="chat-typing hidden">
        <span>Escribiendo</span><span class="dot-pulse">...</span>
    </div>
</div>
</div>

```

6.2 chat-widget.js (lógica del frontend)

```

// chat-widget.js – SOLO se comunica con el backend Java
// NUNCA contiene API keys

document.addEventListener('DOMContentLoaded', () => {
    const chatToggle = document.getElementById('chat-toggle');
    const chatWindow = document.getElementById('chat-window');
    const chatClose = document.getElementById('chat-close');
    const chatInput = document.getElementById('chat-input');
    const chatSend = document.getElementById('chat-send');
    const chatMessages = document.getElementById('chat-messages');
    const chatTyping = document.getElementById('chat-typing');

    // Abrir/Cerrar chat
    chatToggle.addEventListener('click', () => {
        chatWindow.classList.toggle('hidden');
        chatToggle.classList.toggle('hidden');
    });

    chatClose.addEventListener('click', () => {
        chatWindow.classList.add('hidden');
        chatToggle.classList.remove('hidden');
    });

    // Enviar mensaje
    async function enviarMensaje() {
        const texto = chatInput.value.trim();
        if (!texto) return;

        // Mostrar mensaje del usuario
        agregarMensaje('user', texto);
        chatInput.value = '';

        // Mostrar indicador de escritura
    }

```



```

chatTyping.classList.remove('hidden');

try {
  // Enviar a NUESTRO backend Java (no a OpenRouter directamente)
  const respuesta = await fetch('/api/chat', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      pregunta: texto,
      emailUsuario: getEmailUsuario() // opcional
    })
  });

  const datos = await respuesta.json();

  // Ocultar indicador de escritura
  chatTyping.classList.add('hidden');

  // Mostrar respuesta
  if (datos.exito) {
    agregarMensaje('bot', datos.respuesta);
  } else {
    agregarMensaje('bot', 'Lo siento, no pude procesar tu pregunta.');
```

corriendo.');

```

  } catch (error) {
    chatTyping.classList.add('hidden');
    agregarMensaje('bot', 'Error de conexión. Verifica que el servidor esté
    console.error('Error:', error);
  }
}

function agregarMensaje(rol, texto) {
  const div = document.createElement('div');
  div.className = `message ${rol}`;
  div.textContent = texto;
  chatMessages.appendChild(div);
  chatMessages.scrollTop = chatMessages.scrollHeight;
}

function getEmailUsuario() {
  const usuario = localStorage.getItem('user');
  if (usuario) {
    try {
      return JSON.parse(usuario).email || '';
    } catch(e) { return ''; }
  }
  return '';
}

// Event listeners

```

```

chatSend.addEventListener('click', enviarMensaje);
chatInput.addEventListener('keypress', (e) => {
    if (e.key === 'Enter') enviarMensaje();
});
});

```

6.3 chat-widget.css (estilos)

```

/* Widget de chat flotante */
.chat-widget {
    position: fixed;
    bottom: 20px;
    right: 20px;
    z-index: 1000;
    font-family: 'Segoe UI', sans-serif;
}

.chat-toggle-btn {
    background: var(--color-primario, #B59E81);
    color: white;
    border: none;
    padding: 12px 24px;
    border-radius: 25px;
    cursor: pointer;
    font-size: 16px;
    box-shadow: 0 4px 12px rgba(0,0,0,0.2);
    transition: transform 0.2s;
}

.chat-toggle-btn:hover {
    transform: scale(1.05);
}

.chat-window {
    width: 380px;
    height: 520px;
    border-radius: 12px;
    box-shadow: 0 8px 24px rgba(0,0,0,0.15);
    display: flex;
    flex-direction: column;
    overflow: hidden;
}

.chat-window.hidden { display: none; }

.chat-header {
    background: var(--color-primario, #B59E81);
    color: white;
    padding: 14px 18px;
    display: flex;
    justify-content: space-between;
}

```

```
    align-items: center;
    font-weight: bold;
}

.chat-header button {
    background: none;
    border: none;
    color: white;
    font-size: 24px;
    cursor: pointer;
}

.chat-messages {
    flex: 1;
    padding: 16px;
    overflow-y: auto;
    background: #f8f8f8;
}

.message {
    margin-bottom: 12px;
    padding: 10px 14px;
    border-radius: 12px;
    max-width: 85%;
    word-wrap: break-word;
}

.message.user {
    background: var(--color-primario, #B59E81);
    color: white;
    margin-left: auto;
    border-bottom-right-radius: 4px;
}

.message.bot {
    background: white;
    border: 1px solid #e0e0e0;
    margin-right: auto;
    border-bottom-left-radius: 4px;
}

.chat-input-area {
    display: flex;
    padding: 12px;
    background: white;
    border-top: 1px solid #e0e0e0;
    gap: 8px;
}

.chat-input-area input {
    flex: 1;
    padding: 10px;
```

```

border: 1px solid #ccc;
border-radius: 8px;
font-size: 14px;
}

.chat-input-area button {
background: var(--color-primario, #B59E81);
color: white;
border: none;
padding: 10px 20px;
border-radius: 8px;
cursor: pointer;
}

.chat-typing {
padding: 8px 16px;
color: #888;
font-style: italic;
font-size: 13px;
}

.chat-typing.hidden { display: none; }

.dot-pulse::after {
content: '';
animation: dots 1.5s infinite;
}

@keyframes dots {
0%, 20% { content: '.'; }
40% { content: '..'; }
60%, 100% { content: '...'; }
}

```

7. Integración con el Sistema Existente

7.1 System Prompts Especializados

El asesor puede tener **personalidades diferentes** según el contexto:

Contexto	System Prompt	Cuándo se usa
General	"Eres un asesor virtual experto en el sistema inmobiliario Insignia Inmo..."	Por defecto
Inmuebles	"Ayudas a buscar y comparar propiedades. Puedes filtrar por precio, tipo, ubicación..."	Cuando la pregunta menciona propiedades
Clientes	"Ayudas a gestionar clientes, ver su historial de consultas y visitas..."	Cuando se pregunta sobre clientes

Operaciones	"Ayudas con el registro de ventas y arriendos, renovaciones y cancelaciones..."	Cuando se pregunta sobre operaciones
Recomendaciones	"Recomiendas inmuebles basados en preferencias del cliente..."	Cuando se piden recomendaciones

7.2 Posibles Mejoras Post-implementación

Mejora	Descripción	Dificultad
Historial de conversación	Enviar últimos N mensajes como contexto para mantener la coherencia	Media
Streaming de respuesta	Mostrar la respuesta letra por letra (Server-Sent Events)	Alta
Memoria por usuario	Recordar preferencias del usuario entre sesiones usando SQLite	Media
Selección de modelo	Permitir al admin elegir entre Llama, Claude, GPT desde el frontend	Baja
Fallback automático	Si OpenRouter falla, intentar con otro proveedor (Groq, Together)	Media
Moderación de contenido	Filtrar preguntas ofensivas o fuera de contexto	Media
Costos por uso	Llevar conteo de tokens consumidos por usuario/rol	Baja

8. Configuración de Despliegue

8.1 Variable de Entorno

```
# Windows (PowerShell)
$env:OPENROUTER_API_KEY="sk-or-v1-tu-clave-aqui"

# Windows (CMD)
set OPENROUTER_API_KEY=sk-or-v1-tu-clave-aqui

# Linux/Mac
export OPENROUTER_API_KEY="sk-or-v1-tu-clave-aqui"
```

8.2 Archivo .env para desarrollo (opcional)

```
# .env (NUNCA subir a git)
OPENROUTER_API_KEY=sk-or-v1-tu-clave-aqui
```

⚠ **Importante:** El archivo `.env` debe agregarse a `.gitignore` para evitar exponer la clave en el repositorio.

8.3 En el servidor de producción

```
# Configurar como variable de entorno del sistema
# Ejemplo con systemd (Linux):
Environment="OPENROUTER_API_KEY=sk-or-v1-tu-clave-aqui"
```

8.4 Verificación de funcionamiento

```
GET /api/chat/health → { "status": "ok", "modelo": "meta-llama/llama-3-8b-instruct",
"apiKeyConfigurada": true }
```

9. Plan de Trabajo por Fases

Fase 1: Backend Base (Estimación: 2-3 horas)

- ☐ Crear `AIService.java` con integración HTTP a OpenRouter
- ☐ Crear `ChatRequest.java` y `ChatResponse.java` (DTOs)
- ☐ Crear `AsesorService.java` con lógica de contexto
- ☐ Agregar endpoint `POST /api/chat` en `Main.java`
- ☐ Configurar CORS en Javalin
- ☐ Probar con curl/Postman: `POST /api/chat { "pregunta": "¿Cuántas propiedades hay?" }`

Fase 2: Frontend (Estimación: 1-2 horas)

- ☐ Crear `chat-widget.html` (estructura del widget flotante)
- ☐ Crear `chat-widget.css` (estilos del widget)
- ☐ Crear `chat-widget.js` (lógica de fetch al backend)
- ☐ Integrar widget en `index.html` y `landing.html`
- ☐ Probar flujo completo: frontend → backend → OpenRouter → backend → frontend

Fase 3: Pulido y Contexto (Estimación: 2-3 horas)

- ☐ Mejorar `construirContexto()` con datos reales del sistema
- ☐ Implementar System Prompts especializados según detección de intención
- ☐ Agregar animación de "Escribiendo..."
- ☐ Manejo de errores y timeouts
- ☐ Pruebas con preguntas reales de usuarios

Fase 4: Opcionales (Estimación: variable)

- ☐ Streaming de respuestas (SSE)
- ☐ Historial de conversación por sesión
- ☐ Dashboard de uso para admin

10. Diagrama de Secuencia




```
assertNotNull(resp);
assertTrue(resp.isExito());
assertFalse(resp.getRespuesta().isBlank());
}

@Test
void testChatEndpointRechazaVacio() {
    ChatRequest req = new ChatRequest();
    req.setPregunta("");

    // Debería retornar error 400
}
```

Pruebas manuales

```
# Probar endpoint directamente
curl -X POST http://localhost:7070/api/chat \
  -H "Content-Type: application/json" \
  -d '{"pregunta": "¿Qué propiedades hay disponibles en la zona norte?}"'

# Probar health check
curl http://localhost:7070/api/chat/health
```

Documento generado el 27 de mayo de 2026 Proyecto: PropTech / Insignia Inmo Estado: PLAN — Pendiente de aprobación para iniciar implementación